



Đồ án tốt nghiệp Tìm hiểu công nghệ thực tế ảo và ứng dụng (download tại tailieutui)

soạn thảo văn bản (Trường Đại học Trà Vinh)



messages.pdf_cover_qr_code_label

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc nhất tới thầy giáo Vũ Đức Huy, thầy đã tận tình hướng dẫn và giúp đỡ em trong suốt quá trình làm bài tập lớn. Với sự chỉ bảo của thầy, em đã có những định hướng tốt trong việc triển khai và thực hiện các yêu cầu trong quá trình làm bài tập lớn.

Em xin chân thành cảm ơn !

MỤC LỤC

LỜI CẢM ƠN.....	1
MỤC LỤC.....	2
LỜI MỞ ĐẦU.....	4
CHƯƠNG 1. Tổng quan về thực tại ảo.....	5
1.1 Thực tế ảo là gì ?.....	5
1.2 Lịch sử phát triển của công nghệ thực tế ảo.....	5
1.3 Các đặc tính chính của VR.....	6
1.4 Các thành phần một hệ thống VR.....	7
1.4.1 Phần cứng (Hardware).....	7
1.4.2 Phần mềm (Software).....	8
1.5 Các thiết bị cơ bản.....	8
1.5.1 Thiết bị định hướng và chuyển động.....	8
1.5.2 Thiết bị tương tác và phản hồi.....	11
1.6 Một số ứng dụng chính của VR.....	12
1.6.1 Quân sự.....	13
1.6.2 Giáo dục.....	13
1.6.3 Xây dựng.....	14
1.6.4 Y học.....	14
CHƯƠNG 2. Ngôn ngữ VRML.....	16
2.1 Giới thiệu về VRML.....	16
2.1.1 VRML là gì ?.....	16
2.1.2 Định nghĩa về VRML.....	16
2.1.3 Lịch sử ra đời và phát triển của VRML.....	17
2.1.4 Đặc điểm cơ bản của VRML.....	18
2.2 Các vấn đề cơ bản của VRML.....	18
2.2.1 Các thành phần cơ bản của VRML.....	18
2.2.2 Công cụ hiển thị VRML.....	19
2.2.3 Tập tin của VRML.....	19
2.3 Tìm hiểu chi tiết về VRML.....	20
2.3.1 Xây dựng các đối tượng hình học cơ bản.....	20

2.3.2	Xây dựng một số hình phức tạp.....	22
2.3.3	Các phép biến đổi trong VRML.....	28
2.3.4	Màu sắc trong VRML.....	31
2.3.5	Nhóm node.....	32
2.3.6	Một số phương pháp vẽ trong VRML.....	34
2.3.7	Texture Mapping.....	35
2.3.8	Script.....	35
CHƯƠNG 3. Ứng dụng VRML trong việc xây dựng xóm trọ Hoàng Trường		37
3.1	Bài toán.....	37
3.2	Yêu cầu đặt ra và hướng giải quyết.....	37
3.2.1	Xây dựng xóm trọ.....	37
3.2.2	Trang trí xóm trọ.....	38
3.2.3	Xây dựng các đối tượng bên trong xóm trọ.....	39
3.2.4	Di chuyển bên trong xóm trọ.....	39
3.2.5	Chọn đối tượng.....	40
3.2.6	Ánh sáng.....	41
3.2.7	Đóng mở cửa.....	41
3.2.8	Tối ưu khung hình.....	41
3.3	Kết quả đạt được và hướng phát triển.....	42
3.3.1	Kết quả.....	42
PHẦN KẾT LUẬN.....		43
TÀI LIỆU THAM KHẢO.....		44

LỜI MỞ ĐẦU

Thực tế ảo là một thuật ngữ mới xuất hiện khoảng đầu thập kỷ 90, nhưng ở Mỹ và châu Âu thực tế ảo (Virtual Reality) đã và đang trở thành một công nghệ mũi nhọn nhờ khả năng ứng dụng rộng rãi trong mọi lĩnh vực (nghiên cứu và công nghiệp, giáo dục và đào tạo, thương mại và giải trí,..) và tiềm năng kinh tế, cũng như tính lưỡng dụng (trong dân dụng và quân sự) của nó. Tại Việt Nam, tuy là một lĩnh vực mới nhưng đã có những công trình rất hữu ích như: tái hiện lại con Sao La hay một Văn Miếu Quốc Tử Giám ảo mà ta có thể đi lại quan sát trong đó. Chính vì tầm quan trọng cũng như khả năng ứng dụng to lớn đó nên việc nghiên cứu về thực tại ảo là vô cùng cần thiết. Và trên cơ sở đó có thể xây dựng một ứng dụng thực tại ảo hoàn chỉnh.

Chính vì vậy mà em đã chọn đề tài tốt nghiệp: ” Tìm hiểu công nghệ thực tế ảo và ứng dụng”. Đồ án này gồm có phần mở đầu, kết luận và nội dung:

Chương 1: Tổng quan về thực tại ảo

Chương 2: Ngôn ngữ VRML

Chương 3: Ứng dụng VRML trong việc xây dựng xóm trọ Hoàng trường

CHƯƠNG 1. Tổng quan về thực tại ảo

1.1 Thực tế ảo là gì ?

Thực tế ảo-Virtual Reality(VR) là một hệ thống mô phỏng trong đó đồ họa máy tính được sử dụng để tạo ra một thế giới "như thật". Hơn nữa, thế giới "nhân tạo" này không tĩnh tại, mà lại phản ứng, thay đổi theo ý muốn (tín hiệu vào) của người sử dụng (nhờ hành động, lời nói,...). Điều này xác định một đặc tính chính của VR, đó là tương tác thời gian thực (real-time interactivity). Thời gian thực ở đây có nghĩa là máy tính có khả năng nhận biết được tín hiệu vào của người sử dụng và thay đổi ngay lập tức thế giới ảo. Người sử dụng nhìn thấy sự vật thay đổi trên màn hình ngay theo ý muốn của họ và bị thu hút bởi sự mô phỏng này.

Điều này chúng ta có thể nhận thấy ngay khi quan sát trẻ nhỏ chơi video game. Theo báo Bild (Đức), có hai trẻ nhỏ ở Anh bị thu hút và mãi mê chơi Nintendo đến nỗi ngay cả khi nhà chúng đang bị cháy cũng không hề hay biết! Tương tác và khả năng thu hút của VR góp phần lớn vào cảm giác đắm chìm (immersion), cảm giác trở thành một phần của hành động trên màn hình mà người sử dụng đang trải nghiệm. Nhưng VR còn đẩy cảm giác này "thật" hơn nữa nhờ tác động lên tất cả các kênh cảm giác của con người. Trong thực tế, người dùng không những nhìn thấy đối tượng đồ họa 3D nổi (như hình nổi ở trang cuối báo Hoa học trò đã đăng trước kia), điều khiển (xoay, di chuyển,...) được đối tượng trên màn hình (như trong game), mà còn sờ và cảm thấy chúng như có thật. Ngoài khả năng nhìn (thị giác), nghe (thính giác), sờ (xúc giác), các nhà nghiên cứu cũng đã nghiên cứu để tạo các cảm giác khác như ngửi (khứu giác), nếm (vị giác). Tuy nhiên hiện nay trong VR các cảm giác này cũng ít được sử dụng đến.

Từ các phân tích trên, chúng ta có thể thấy định nghĩa sau đây của C. Burdea và P. Coiffet về VR là tương đối chính xác: VR- Thực Tế Ảo là một hệ thống giao diện cấp cao giữa Người sử dụng và Máy tính. Hệ thống này mô phỏng các sự vật và hiện tượng theo thời gian thực và tương tác với người sử dụng qua tổng hợp các kênh cảm giác. Đó là ngũ giác gồm: thị giác, thính giác, xúc giác, khứu giác, vị giác.

1.2 Lịch sử phát triển của công nghệ thực tế ảo

Khái niệm thực tế ảo đã có trong nhiều thập niên nhưng nó chỉ thực sự được nhận thức vào đầu những năm 90. Vào giữa những năm 50 Morton Heilig (Mỹ) đã phát minh ra thiết bị mô phỏng SENSORAMA. Đó là 1 thiết bị điều khiển 1 người sử dụng gồm có : một màn hình thực thể kính, quạt, máy tạo mùi, loa âm thanh và 1 chiếc ghế có thể di chuyển được. Ông cũng phát minh ra màn hình truyền hình được gắn vào đầu để có thể xem phim 3D. Tuy là những sản phẩm phục vụ cho điện ảnh nhưng những khái niệm của Heilig đã trở thành tiền đề cho VR sau này.

Những kỹ sư của Công ty Philco là những người đầu tiên phát triển HMD vào 1961, gọi là Headsight. Cái mũ sắt bao gồm một màn ảnh và hệ thống theo dõi video đã những kỹ sư liên kết tới một hệ thống camera mạch đóng. Họ dự định sử dụng HMD trong các tình huống nguy hiểm - một người có thể quan sát một môi trường thực sự từ xa, điều chỉnh góc quay camera bằng cách quay đầu. Bell Laboratories đã sử dụng HMD cho những phi công lái máy bay trực thăng. Họ liên kết HMD với những camera hồng ngoại gắn bên ngoài máy bay giúp phi công có thể nhìn rõ ngay cả trong môi trường thiếu ánh sáng.

Vào 1965, một nhà khoa học máy tính có tên Ivan Sutherland hình dung điều mà ông ta gọi là "Ultimate Display". Sử dụng hiển thị này, một người có thể thấy một thế giới ảo hiện ra như thế giới vật lý thật. Điều này đã định hướng toàn bộ tầm nhìn về VR. Khái niệm của Sutherland bao gồm :

- ❖ Một thế giới ảo mà ta có thể quan sát thông qua một HMD
- ❖ Một máy tính để duy trì các mô hình trong thời gian thực
- ❖ Các khả năng cho người sử dụng để thao tác những đối tượng thực tế một cách trực quan nhất.

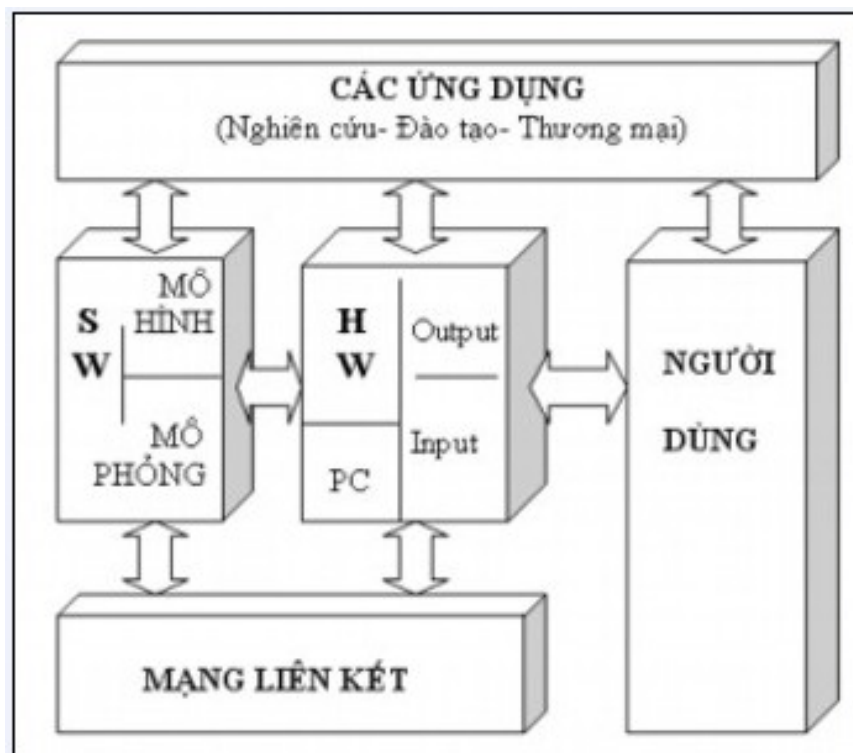
1.3 Các đặc tính chính của VR

Như trên đã trình bày, 2 đặc tính chính của VR là Tương tác và Đắm chìm, đây là hai "I" (Interactive, Immersion) mà nhiều người đã biết. Tuy nhiên VR cần có 1 đặc tính thứ 3 mà ít người để ý tới. VR không chỉ là một hệ thống tương tác Người- Máy tính, mà các ứng dụng của nó còn liên quan tới việc giải quyết các vấn đề thật trong kỹ thuật, y

học, quân sự,...Các ứng dụng này do các nhà phát triển VR thiết kế, điều này phụ thuộc rất nhiều vào khả năng Tưởng tượng của con người, đó chính là đặc tính "I" (Imagination) thứ 3 của VR. Do đó có thể coi VR là tổng hợp của 3 yếu tố: Tương tác- Đắm chìm- Tưởng tượng, (3 I trong tiếng Anh: Interactive- Immersion- Imagination)

1.4 Các thành phần một hệ thống VR

Tổng quát một VR bao gồm những thành phần sau



Hình 1.1 Các thành phần của một hệ thống VR

1.4.1 Phần cứng (Hardware)

Phần cứng của một VR bao gồm:

- ❖ Máy tính (PC hay Workstation với cấu hình đồ họa mạnh).
- ❖ Các thiết bị đầu vào (Input devices): Bộ dò vị trí (position tracking) để xác định vị trí quan sát. Bộ giao diện định vị (Navigation interfaces) để di chuyển vị trí người sử dụng. Bộ giao diện cử chỉ (Gesture interfaces) như găng tay dữ liệu (data glove) để người sử dụng có thể điều khiển đối tượng.
- ❖ Các thiết bị đầu ra (Output devices): gồm hiển thị đồ họa (như màn hình, HDM,..) để nhìn được đối tượng 3D nổi. Thiết bị âm thanh (loa) để nghe được

âm thanh vòm (như Hi-Fi, Surround,..). Bộ phản hồi cảm giác (Haptic feedback như căng tay,..) để tạo xúc giác khi sờ, nắm đối tượng. Bộ phản hồi xung lực (Force Feedback) để tạo lực tác động như khi đạp xe, đi đường xóc,...

1.4.2 Phần mềm (Software)

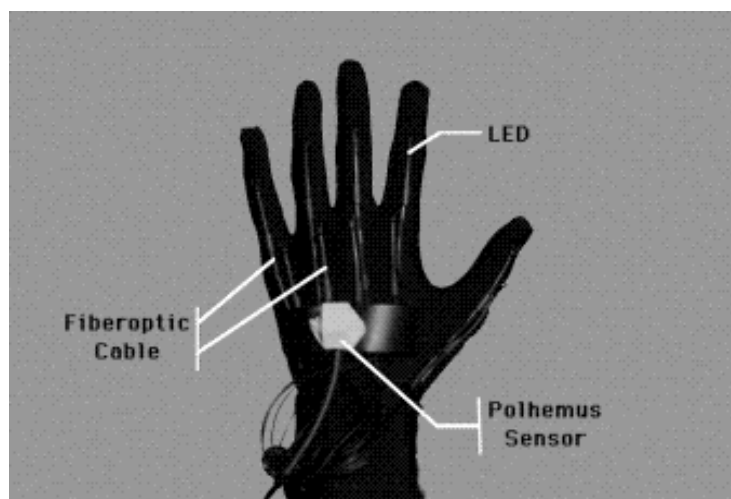
Phần mềm luôn là linh hồn của VR cũng như đối với bất cứ một hệ thống máy tính hiện đại nào. Về mặt nguyên tắc có thể dùng bất cứ ngôn ngữ lập trình hay phần mềm đồ họa nào để mô hình hóa (modelling) và mô phỏng (simulation) các đối tượng của VR. Ví dụ như các ngôn ngữ (có thể tìm miễn phí) OpenGL, C++, Java3D, VRML, X3D,.. hay các phần mềm thương mại như WorldToolKit, PeopleShop,... Phần mềm của bất kỳ VR nào cũng phải bảo đảm 2 công dụng chính: Tạo hình và Mô phỏng. Các đối tượng của VR được mô hình hóa nhờ chính phần mềm này hay chuyển sang từ các mô hình 3D (thiết kế nhờ các phần mềm CAD khác như AutoCAD, 3D Studio,..). Sau đó phần mềm VR phải có khả năng mô phỏng động học, động lực học, và mô phỏng ứng xử của đối tượng.

1.5 Các thiết bị cơ bản

1.5.1 Thiết bị định hướng và chuyển động

DataGloves

Thiết bị đo lường bàn tay phải cảm nhận được cả độ cong của các ngón tay và vị trí, sự định hướng của cổ tay trong thời gian thực. Thiết bị thương mại đầu tiên là DataGloves từ viện nghiên cứu VPL. DataGloves bao gồm 1 găng tay nylon nhẹ có các cảm biến quang học được gắn ở các ngón tay.



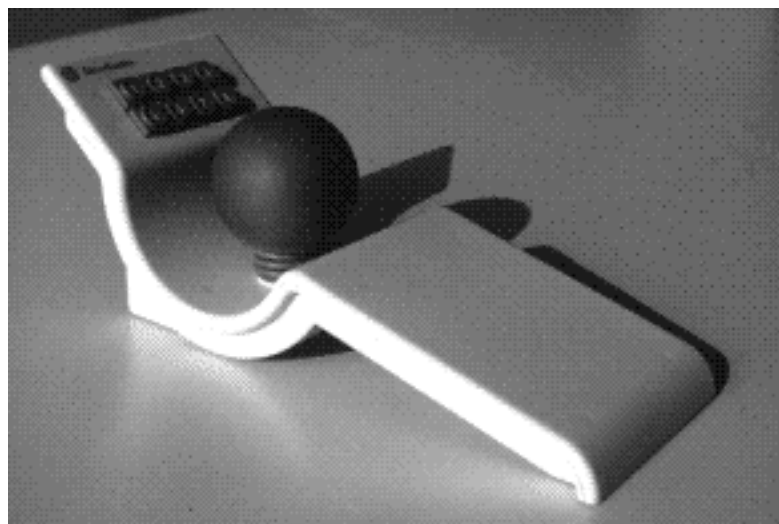
Hình 1.2 DataGloves

3D Mouse and SpaceBall



Hình 1.3 3D Mouse và SpaceBall

Chuột Logitech 3D dựa trên một mảng các vị trí siêu âm tham chiếu, đó là 1 cái kiềng gồm 3 loa siêu âm đặt ở 3 góc tam giác phát ra tín hiệu siêu thanh. Nó được sử dụng để theo dõi thiết bị thu, định hướng và chuyển động. Nó qui định thành phần của tỷ lệ gửi ra trong tất cả 6. mức tự do: X, Y, Z, Pitch, Yaw, và Roll.



Hình 1.4 Mouse

Shutter glasses



Hình 1.5 Shutter glasses

Head-Mounted Displays

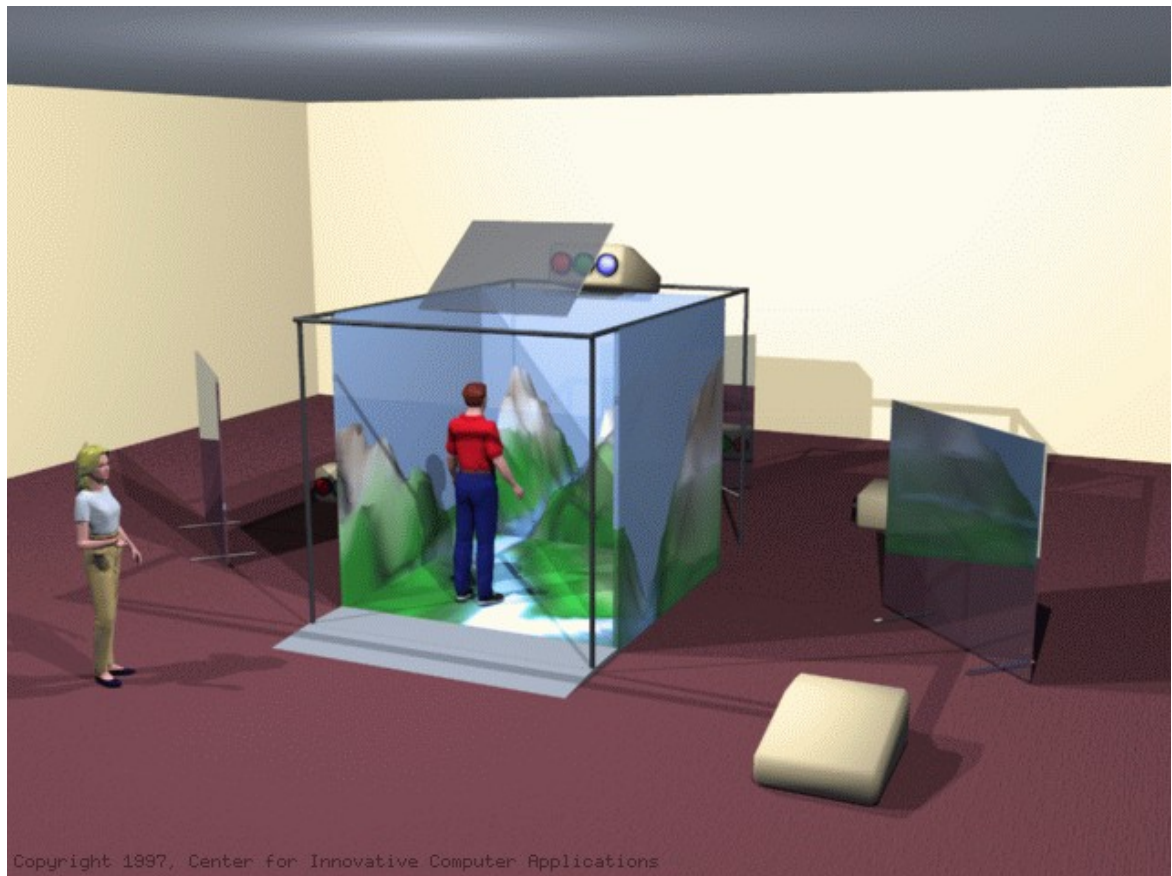


Hình 1.6 Head-Mounted Displays

Cave

CAVE là 1 nhà hát có kích thước 10 X 10 X 9 được đặt bên trong 1 phòng lớn hơn có kích thước 35 X 25 X 13. Phòng bên ngoài phải được chiếu sáng trong suốt quá trình sử dụng CAVE. Các bức tường của CAVE được tạo bởi các màn chiếu và sàn nhà cũng là một màn chiếu thẳng đứng. Máy chiếu độ phân giải cao hiển thị hình ảnh lên toàn bộ những màn ảnh khác bằng các tấm gương phản chiếu. Người dùng sẽ đi vào bên trong CAVE và đeo 1 chiếc kính đặc biệt để có thể nhìn thấy những hình ảnh 3 chiều mà CAVE hiển thị. Với những chiếc kính này người dùng có thể thấy các đối tượng thực sự

nổi trong không khí và có thể đi lại xung quanh chúng. Điều này là hoàn toàn khả dĩ với các cảm biến điện tử. Khung của CAVE được làm từ i-nox không từ tính để có thể can thiệp một cách tốt nhất vào các cảm biến điện tử. Khi một người đi lại trong CAVE, chuyển động của họ được theo dõi bởi các cảm biến này và video sẽ điều chỉnh cho phù hợp. Máy tính sẽ kiểm soát việc này của CAVE cũng như cả khía cạnh âm thanh nữa. Có rất nhiều loa được đặt trong CAVE dưới nhiều góc độ giúp cho không chỉ có hình ảnh 3 chiều mà có cả âm thanh 3 chiều nữa.



Hình 1.7 Cave

1.5.2 Thiết bị tương tác và phản hồi

Các thiết bị này cảm nhận một số nhân tố sau của thiết bị khác gây ra: nhiệt độ, vận tốc di chuyển, sự chuyển động, áp lực và các ngoại lực khác.

CyberTouch



Hình 1.8 CyberTouch

CyberGrasp



Hình 1.9 CyberGrasp

1.6 Một số ứng dụng chính của VR

Tại các nước phát triển, chúng ta có thể nhận thấy VR được ứng dụng trong mọi lĩnh vực: Khoa học kỹ thuật, kiến trúc, quân sự, giải trí,... và đáp ứng mọi nhu cầu: Nghiên cứu- Giáo dục- Thương mại. Y học là lĩnh vực ứng dụng truyền thống của VR. Bên cạnh đó VR cũng được ứng dụng trong giáo dục, nghệ thuật, giải trí. Trong lĩnh vực quân sự, VR cũng được ứng dụng rất nhiều ở các nước phát triển. Bên cạnh các ứng dụng truyền thống ở trên, cũng có một số ứng dụng mới nổi lên trong thời gian gần đây của VR như: VR ứng dụng trong sản xuất, VR ứng dụng trong ngành rôbốt, VR ứng

dụng trong hiển thị thông tin (thăm dò dầu mỏ, hiển thị thông tin khối,...) VR có tiềm năng ứng dụng vô cùng lớn. Có thể nói tóm lại một điều: Mọi lĩnh vực "có thật" trong cuộc sống đều có thể ứng dụng "thực tế ảo" để nghiên cứu và phát triển hoàn thiện hơn.

1.6.1 Quân sự

Với việc phát triển của VR, các binh sĩ sẽ được huấn luyện 1 cách trực quan nhất các kỹ năng cần thiết như : lái máy bay, lái xe tăng, ... trước khi tham gia công việc thực tế. Điều này vừa bảo đảm an toàn cho binh sĩ, vừa tiết kiệm được chi phí cho các khóa huấn luyện thực tế. Lầu Năm Góc vừa đưa ra quyết định sẽ đầu tư 36 triệu USD cho quân đội Mỹ để phát triển một game đặc biệt nhằm huấn luyện binh sĩ chống lại khủng bố dưới dạng chiến thuật thực tế ảo. Với hệ thống trò chơi đặc biệt này, những binh sĩ có thể tập luyện những bài tập của mình ngay tại nhà nhằm chống lại những tình huống có thể phát sinh ra trong thực tế. Đây sẽ là một game rất sống động, có tính hành động cao với môi trường và bối cảnh bám sát với thực tế. Những người lính sẽ phải vận dụng tất cả những kỹ năng đã được rèn giũa trong quân đội.



Hình 1.10 Binh lính học nhảy dù bằng thực tế ảo

1.6.2 Giáo dục

Ở các nước phương Tây việc ở nhà học qua Internet không còn là điều mới mẻ nữa. Và công nghệ VR sẽ làm cho việc này trở nên thú vị hơn rất nhiều. Giống như một

game MMORPG bạn điều khiển 1 nhân vật đại diện cho bạn đi lại trong 1 trường học ảo được xây dựng trên máy tính. Bạn có thể tham gia vào bất cứ lớp học ảo nào mà bạn thích, nói chuyện với những thành viên khác trong lớp.



Hình 1.11 Cảnh trong một lớp học ảo

1.6.3 Xây dựng

Bạn muốn xây nhà. Bạn thuê một kiến trúc sư thiết kế cho ngôi nhà tương lai của bạn. Anh ta hoàn thành nó trên bản vẽ và liệu bạn có thể tưởng tượng ra nó thế nào không? Có thể nhưng chắc là không thể chính xác được. Và khi hoàn thành thì chưa chắc nó đã đúng ý của bạn. Giờ đây ngôi nhà đó được xây dựng trên máy tính, bạn có thể đi lại khắp nơi trong nhà, xem xét từng ngõ ngách nhỏ nhất.

1.6.4 Y học

Thực tại ảo giải quyết được rất nhiều vấn đề trong y học: cung cấp môi trường thực hành cho nghiên cứu và học tập, rất hữu ích trong việc mô phỏng các ca phẫu thuật tránh gây rủi ro trong thực tế

Như vậy thực tại ảo có ứng dụng trong hầu hết các lĩnh vực của cuộc sống. Qua đó cũng nhận thấy được ý nghĩa to lớn của việc ứng dụng thực tại ảo, bởi những vấn đề

khó khăn mà nếu không có thực tại ảo thì rất khó giải quyết hoặc hiệu quả không cao mà chi phí tốn kém.

CHƯƠNG 2. Ngôn ngữ VRML

2.1 Giới thiệu về VRML

2.1.1 VRML là gì ?

VRML (Virtual Reality Modeling Language) là ngôn ngữ mô hình hóa thực tế ảo, một định dạng tập tin được sử dụng trong việc mô tả các thể giới và các đối tượng đồ họa tương tác ba chiều. VRML được thiết kế dùng trong môi trường Internet, Intranet và các hệ thống máy khách cục bộ (local client). VRML còn được dự trù trở thành một chuẩn trao đổi đa năng cho đồ họa ba chiều tích hợp và truyền thông đa phương tiện. VRML có thể được sử dụng trong rất nhiều lĩnh vực ứng dụng chẳng hạn như trực quan hóa các khái niệm khoa học và kỹ thuật, trình diễn đa phương tiện, giải trí và giáo dục, hỗ trợ web và chia sẻ các thể giới ảo.

Ra đời phiên bản đầu tiên vào tháng 10 năm 1994 (VRML 1.0), cho đến nay VRML đã phát triển tới phiên bản 2.0 với các chức năng mạnh mẽ, nhanh chóng trở thành chuẩn phát triển cho nhiều chương trình đồ họa. VRML là ngôn ngữ Internet 3D dùng để phát triển đồ họa 3D trên Web, có cấu trúc chặt chẽ, với khả năng mạnh mẽ, giúp cho việc xây dựng các ứng dụng 3D một cách nhanh chóng và chân thực nhất.

2.1.2 Định nghĩa về VRML

Ngôn ngữ VRML là ngôn ngữ sử dụng mô hình phân cấp trong việc thể hiện các tương tác với các đối tượng của mô hình, VRML được sử dụng để phát triển những hình ảnh 3D và khung cảnh trên Web. Các file VRML có kích thước nhỏ, thường không quá 1Mb.

Ngôn ngữ mô hình hóa thực tại ảo VRML là một chuẩn không chính thức để mô tả thực tế ảo mà không phụ thuộc vào hệ điều hành thông qua Internet. Chỉ với một file text bạn có thể mô tả, tương tác, điều khiển một thể giới ảo mà không bị hạn chế nhiều.

VRML cho phép truyền đi trong mạng những hình ảnh 3D. Với kích thước khá nhỏ so với băng thông, phần lớn giới hạn trong khoảng 100 - 200Kb nên các file VRML được truyền đi một cách khá dễ dàng. Nếu HTML là định dạng văn bản thì VRML là định dạng đối tượng 3D. Hiện nay VRML có lợi thế là sự đơn giản, hỗ trợ dịch vụ web3D.

2.1.3 Lịch sử ra đời và phát triển của VRML

VRML đã trở thành một ngôn ngữ chuẩn cho việc mô phỏng tương tác thế giới 3D trên Web. Với mục đích xây dựng định dạng chuẩn cho phép mô tả thế giới thực trên máy tính mà cho phép chạy trên môi trường web, VRML đã trở thành chuẩn ISO từ năm 1997.

VRML ra đời vào mùa xuân năm 1994 ở hội nghị WWW được tổ chức đầu tiên tại Gieneva, Thụy Sĩ. Tim Berners-Lee và Dave Raggett đã tổ chức ra phiên họp có tên là Birds of a Feather (BOF) để mô tả giao diện thực tế ảo trên WWW. Nhiều thành viên tham dự phiên họp BOF đã mô tả nhiều dự án thực hiện việc xây dựng các công cụ hiển thị đồ họa 3D cho phép có nhiều thao tác hữu ích trên Web. Những thành viên này đã nhất trí đồng ý sự cần thiết cho các công cụ này có một ngôn ngữ chung, phổ biến cho định dạng, xác định việc mô tả thế giới 3D và các siêu liên kết WWW. Vì thế cụm từ “the Virtual Reality Markup Language” ra đời, từ Markup sau đó đã được đổi thành Modelling để phản ánh bản chất tự nhiên của VRML.

Sau phiên họp BOF một thời gian ngắn thì tổ chức www-vrml mailing list được thành lập để tập trung vào xây dựng phiên bản VRML đầu tiên. Sự hưởng ứng lời mời của tổ chức này kéo dài đến một tuần và có trên một nghìn khách mời tham dự. Tại buổi họp chủ tịch Mark Pesce đã thông báo ý kiến của mình là đưa ra phiên bản phác thảo xây dựng VRML đã có sẵn ở hội nghị mùa xuân năm 1994 được tổ chức mới cách đó 5 tháng. Bản phác thảo của Mark Pesce đã có được sự đồng ý chung.

Vào tháng 3/ 1995 thì có một thảo luận trên www-vrml mailing list liên quan đến việc tạo ra những tương tác của người sử dụng với hoạt cảnh và tất cả mọi người đều đi đến thống nhất ý kiến những thứ mới đưa ra đó thực sự cần thiết cho VRML. Công ty Silicon Graphics cộng tác với hãng Sony Research và Mitra để đưa ra phiên bản mới cho VRML. Bản đề trình của Silicon Graphics có tên là Moving Worlds đến tổ chức Request for Proposals cho việc xây dựng phiên bản mới VRML, bản đề trình này là một minh chứng cho sự cộng tác thành công của tất cả các thành viên của Silicon Graphics, Sony và Mitra. Năm 1996 tại New Orleans, phiên bản đầu tiên của VRML 2.0 được đưa ra.

Vào tháng 7/1996 tổ chức tiêu chuẩn quốc tế (ISO) đã thông nhất ý kiến phiên bản năm 1996 của VRML 2.0 như là một đề xuất mà sẽ được đưa ra xem xét vào tháng

4/1997. Sau khi bỏ phiếu về chuẩn ISO thì VRML97 được đưa ra như một chuẩn ISO vào năm 1997.

2.1.4 Đặc điểm cơ bản của VRML

Tiêu chuẩn cho việc xác định đối tượng 3D, quang cảnh và cho sự liên kết các mô hình với nhau là:

- ❖ Không phụ thuộc phần cứng: có thể chạy trên các máy tính do các nhà sản xuất khác nhau chế tạo.
- ❖ Có thể mở rộng: có thể chấp nhận các lệnh mới do người sử dụng thêm vào hoặc quy định.
- ❖ Thao tác được thế giới ảo thông qua môi trường Internet có băng thông thấp.

Cùng với VRML thì có thể xem thông tin trong mô hình 3D trên Internet. VRML được thiết kế dành riêng cho việc hiển thị thế giới 3D và không phải là sự mở rộng của HTML. HTML có khả năng hiển thị các đối tượng tĩnh và động, các đối tượng multimedia cùng với các siêu liên kết khác đến các media khác như là văn bản, âm thanh, phim và hình ảnh.

2.2 Các vấn đề cơ bản của VRML

2.2.1 Các thành phần cơ bản của VRML

Gồm các trình duyệt (Browsers) và bộ soạn thảo dành riêng cho ngôn ngữ VRML. Các file chỉ có thể đọc nếu hệ thống có trình duyệt VRML.

Trình duyệt VRML cũng giống như trình duyệt Internet(Internet Explorer hay Fire Fox) và được tích hợp trong các trình duyệt này.

Bộ soạn thảo VRML cho phép người dùng gõ mã VRML. Hiện nay có nhiều bộ soạn thảo nhưng VRML Pad là khá thông dụng khi có thể cho xem trực tiếp kết quả mà không cần qua trình duyệt Internet.

2.2.2 Công cụ hiển thị VRML

Để hiển thị các file VRML thì ta sử dụng chương trình Cortona VRML Client của hãng Parallrl Graphics. Chương trình sẽ giúp bạn thuận tiện hơn khi xem các mô hình ảo trên máy tính một cách trực quan sinh động.

Yêu cầu trước khi cài đặt Cortona VRML Client:

- Hệ điều hành Microsoft® Windows® ME/2000/XP
- Trình duyệt Web Internet Explorer 6.0 trở lên, Netscape Navigator 8.0 trở lên, Mozilla Firefox 1.5 trở lên, Opera 8.5 trở lên
- CPU Pentium® II 300 MHz trở lên.
- RAM tối thiểu 64 MB.
- Độ phân giải màn hình tối thiểu 1024x768.
- Card đồ họa hỗ trợ 3D và cài đặt DirectX 9

Cortona VRML Client tương thích với hầu hết các trình duyệt như Internet Explorer, Netscape Browser, Mozilla, Mozilla Firefox và các công cụ văn phòng như Word, PowerPoint...

Tính năng của Cortona VRML ClientCortona VRML Client sẽ trình diễn toàn bộ mô hình 3D trên máy tính một cách hoàn hảo như khi người tạo ra đó. ứng dụng hoàn chỉnh trên toàn bộ hiệu ứng trên nhiều hệ thống như Flash, DirectX9, MPEG4... Khi bạn truy xuất vào một ứng dụng VRML, toàn bộ hình mô phỏng sẽ được trình diễn tương tác trên nền 3D dạng mở. Rất ấn tượng và bắt mắt.

2.2.3 Tập tin của VRML

Tập tin của VRML có thể tạo ra từ nhiều cách: dùng một trình soạn thảo văn bản như Nopepad để soạn thảo, sau đó lưu file với phần mở rộng là. wrl. Hoặc ta có thể soạn thảo trên một trình soạn thảo dành riêng cho VRML như VRML Pad và chạy trực tiếp ứng dụng.

Một file VRML gồm có các phần như: header, scene graph, prototype và event routing.

- ❖ **Header:** dùng để nhận dạng tập tin VRML và cách mã hóa. Header của file VRML bắt đầu bằng dấu #. Ngoài lần xuất hiện đầu tiên ra thì dấu # đánh dấu những gì theo sau nó là phần chú thích. File tiêu đề của VRML có dạng: #VRML V1.0 ascii dành riêng cho phiên bản VRML 1.0 và #VRML V2.0 utf-8 dành cho phiên bản 2.0.
- ❖ **Scene Graph:** chứa những node mô tả các đối tượng và các thuộc tính đi kèm. Nó gần như một cây phả hệ gồm các nhóm đối tượng.
- ❖ **Prototype:** cho phép một tập các nút kiểu VRML được mở rộng bởi người sử dụng. Các định danh kiểu này có thể được bao hàm trong file (mà chúng được sử dụng) hay định nghĩa ở bên ngoài (file đó).
- ❖ **Event routing:** một số nút có thể phát sinh những sự kiện đáp trả những thay đổi môi trường do tương tác phía người dùng. “Event routing” cho phép một sự kiện phát sinh được truyền đến các “đích”- những nút trong hệ thống, từ đó gây ra những thay đổi cho riêng nút đó và hệ thống

2.3 Tìm hiểu chi tiết về VRML

2.3.1 Xây dựng các đối tượng hình học cơ bản

Một thế giới VRML được cấu tạo nên từ các đối tượng hình học như: hình hộp, hình trụ, hình nón, hình cầu, văn bản.

Shape là thẻ mà các đối tượng trong nó đều được hiển thị. Trong node Shape có node con là geometry chỉ rõ dạng hình học được vẽ ra thế nào. Node geometry có các thuộc tính:

- ❖ **Box** - hình hộp

Ví dụ :

```
Shape{
  geometry Box{ size 2.0 2.0 2.0 }
}
```

Các tham số

- size X Y Z (chiều rộng, chiều cao, chiều sâu)

❖ **Cone** - hình nón

Ví dụ :

```
Shape{
  Geometry Cone{
    height 2.0
    bottomRadius 1.0
    bottom TRUE
    side TRUE
  }
}
```

Các tham số

- height X : chiều cao của hình nón
- bottomRadius Y : bán kính của đáy
- bottom TRUE / FALSE : hiện / ẩn đáy
- side TRUE / FALSE : hiện / ẩn mặt bên

❖ **Cylinder** : vẽ hình trụ

Ví dụ :

```
Shape{
  Geometry Cylinder{
    height 2.0
    radius 1.0
    bottom TRUE
    top TRUE
    side TRUE
  }
}
```

Các tham số

- height X : chiều cao
- radius Y : bán kính mặt đáy
- bottom TRUE / FALSE : hiện / ẩn mặt đáy
- top TRUE / FALSE : hiện / ẩn mặt nắp

- side TRUE / FALSE : hiện / ẩn các mặt bên

❖ **Sphere** : vẽ hình cầu

Ví dụ :

```
Shape{
  Geometry Cylinder{
    Radius 1.0
  }
}
```

Các tham số

- Radius X : bán kính

2.3.2 Xây dựng một số hình phức tạp

❖ **Text** : văn bản

Cú pháp

```
Shape{
  Geometry Text{
    String ["Text","Shape"]
    fontStyle FontStyle{
      family "SERIF"
      style "BOLD"}
  }
}
```

Các tham số

- string ["Nội dung văn bản"]
- family SERIP / SANS / TYPEWRITER
- style BOLD / ITALIC / BOLDITALIC / PLAIN
- size X : chiều cao của chữ
- spacing Y :
- justify FIRST / BEGIN / MIDDLE / END
- horizontal TRUE / FALSE : trình bày ngang / dọc màn hình
- leftToRight TRUE / FALSE : trình bày từ phải / trái sang
- topToBottom TRUE / FALSE : trình bày từ trên / dưới

Ví dụ

```
Shape{
  Geometry Text{
    String ["Hi all, Today is fine."]
    fontStyle FontStyle{
      family "TYPEWRITER"
      style "plain"
      horizontal TRUE
      justify "MIDDLE"
      leftToRight TRUE
      size 0.9
      spacing 1.0
      topToBottom TRUE
    }
  }
  Lenght [6]
  maxExtent 6
}
```

❖ Xây dựng các đường thẳng trong hệ tọa độ ba chiều

Thẻ IndexedLineSet xác định một tập hợp các đường thẳng trong hệ tọa độ không gian ba chiều của thế giới VRML và tập hợp các màu tương ứng cho các đường thẳng đó.

Thẻ này có các thuộc tính:

- **Coord và color:** số thành phần trong trường color không nhất thiết phải bằng với số thành phần trong trường coord. Chẳng hạn khi colorPerVexter nhận giá trị là FALSE.
- **CoordIndex:** bao gồm một dãy chỉ số thứ tự cho các điểm ảnh tạo nên đường thẳng. Ví dụ colorIndex [0 1 2 3] có nghĩa là đường thẳng được tạo bởi điểm thứ nhất nối với điểm thứ hai, thứ hai nối với thứ ba, thứ ba nối với thứ tư, các điểm ảnh được xác định trong thẻ Coordinate. Một ví dụ khác coordIndex [0 1 -1 2 3] ở đây kí hiệu -1 cho biết hình ảnh được tạo trong VRML gồm hai đường thẳng: một đường thẳng tạo bởi điểm thứ nhất nối với điểm thứ hai và một đường thẳng khác tạo bởi điểm thứ ba nối với thứ tư.

- **ColorIndex:** được sử dụng khi colorPerVexter nhận giá trị FALSE, khi colorPerIndex nhận giá trị TRUE thì có thể bỏ qua trường colorIndex trong thẻ IndexedLineSet bởi vì nếu colorPerIndex nhận giá trị TRUE thì bắt buộc phải thiết lập màu cho mỗi điểm ảnh.
- **ColorPerVexter:** có giá trị boolean. Khi colorPerVexter nhận giá trị TRUE thì màu của đường thẳng sẽ là màu trung bình cả hai màu tại hai điểm ảnh tạo nên đường thẳng.

Cú pháp

```
IndexedLineSet{
  Coord coordinate[]
  coordIndex
  color Color[]
  colorIndex[]
  colorPerVexter TRUE
}
```

Ví dụ

```
Shape{
  Appearance Appearance{}
  Geometry IndexedLineSet{
    Coord Coordinate{
      Point[
        0.0 0.0 0.0,0.0 4.0 0.0, 2.0 0.0 0.0,]
      }
    Color Color{
      Color[
        1.0 1.0 1.0,1.0 0.0 0.0,0.0 0.0 1.0
      ]
    }
    coordIndex[
      0 1 2 0
    ]
    colorPerVexter TRUE
  }
}
```

❖ Xây dựng khung bề mặt trong không gian

Thẻ IndexedFaceSet sẽ tạo lên bề mặt bằng cách kết hợp các điểm với nhau.

Cú pháp

```
IndexedFaceSet
{
    Coord Color[]
    coordIndex[]
    color color[]
    colorIndex[]
    colorPerVexter TRUE
    convex TRUE
    solid TRUE
}
```

- Coord, coordIndex, color, colorIndex, colorPerVexter có đặc điểm tương tự như đã nói ở trên. Chú ý là bề mặt luôn luôn được xác định bởi các đường thẳng khép kín vì thế không phải chỉ ra điểm đầu tiên lại một lần nữa trong trường coord.

Ví dụ

```
Coord Coordinate
{
    Point [0 0 0,1 0 0,1 1 0,0 1 0]
}
```

Đã có bốn điểm xác định, khi liên kết các điểm này sẽ tạo ra hình vuông bằng cách sử dụng thuộc tính coordIndex[] và coordIndex[0 1 2 3] nói rằng điểm thứ nhất nối với điểm thứ hai, điểm thứ hai nối với điểm thứ ba, điểm thứ ba nối với điểm thứ tư và điểm thứ tư nối với điểm thứ nhất.

- Convex định nghĩa bề mặt là lồi hay lõm, trường này có giá trị kiểu boolean. Bộ trình duyệt VRML chỉ vẽ bề mặt lồi, trong trường hợp vẽ bề mặt lõm, bộ trình duyệt chia bề mặt đó thành các bề mặt lồi bé hơn để vẽ.

Ví dụ

```
Shape
{
```

```

Appearance appearance {}
Geometry IndexedFaceSet
{
    Coord coordinate
    {
        Point[0 0 0,1 1 0,0 1 0,1 1 0,1 0 0,1 0 1]
    }
    coordIndex[0 1 2 3 -1 4 5 6]
    color Color
    {
        Color[1 1 1,1 0 0]
    }
    colorIndex [0 1]
    colorPerVertex FALSE
    convex FALSE
    solid FALSE
}
}

```

❖ Xây dựng khung lưới và bản đồ trong không gian

Thẻ ElevationGrid cho phép xây dựng khung lưới được tạo bởi các điểm có độ cao xác định trong không gian. Thẻ này rất hữu ích cho việc xây dựng các mạng lưới hoặc địa hình.

Hình ảnh xây dựng được đặt trong không gian của mặt phẳng OXYZ. Điểm bắt đầu là gốc tọa độ, các điểm còn lại tạo nên lưới phải nằm theo hướng dương của các trục OX và OZ.

Cú pháp

```

ElevationGrid
{
    xDimension 0
    xSpacing 0.0
    zDimension 0
    zSpacing 0.0
}

```

```

        height[]
        color color[]
        colorPerVexter TRUE
        convex TRUE
        solid TRUE
    }

```

Tham số

- Color, colorPerVexter, convex, solid có tính chất tương tự như đã nói ở phần trên.
- xDimension chứa số điểm bên trong lưới nằm trên trục X.
- zDimension chứa số điểm bên trong lưới nằm trên trục Z.
- xSpacing là khoảng cách của hai điểm liên tiếp nhau theo hướng của trục X.
- zSpacing là khoảng cách của hai điểm liên tiếp nhau theo hướng của trục Z.
- height chứa một danh sách các giá trị độ cao của mỗi điểm trong lưới.

Các điểm này được tính theo thứ tự từ trái sang phải từ trên xuống dưới.

Ví dụ

```

Shape {
  appearance Appearance {
    material Material { }
  }
  geometry ElevationGrid {
    xDimension 9
    zDimension 9
    xSpacing 1.0
    zSpacing 1.0
    solid FALSE
    height [
      0.0, 0.0, 0.5, 1.0, 0.5, 0.0, 0.0, 0.0, 0.0,
      0.0, 0.0, 0.0, 0.0, 2.5, 0.5, 0.0, 0.0, 0.0,
      0.0, 0.0, 0.5, 0.5, 3.0, 1.0, 0.5, 0.0, 1.0,
      0.0, 0.0, 0.5, 2.0, 4.5, 2.5, 1.0, 1.5, 0.5,

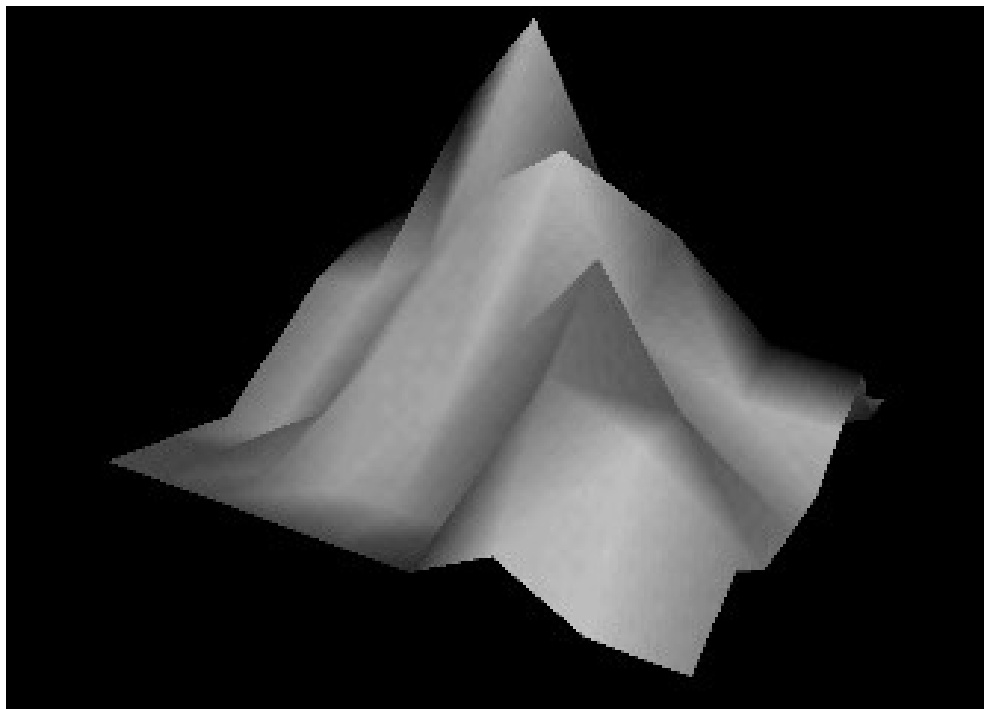
```

```

1.0, 2.5, 3.0, 4.5, 5.5, 3.5, 3.0, 1.0, 0.0,
0.5, 2.0, 2.0, 2.5, 3.5, 4.0, 2.0, 0.5, 0.0,
0.0, 0.0, 0.5, 1.5, 1.0, 2.0, 3.0, 1.5, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 2.0, 1.5, 0.5,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.5, 0.0, 0.0,
]
}
}

```

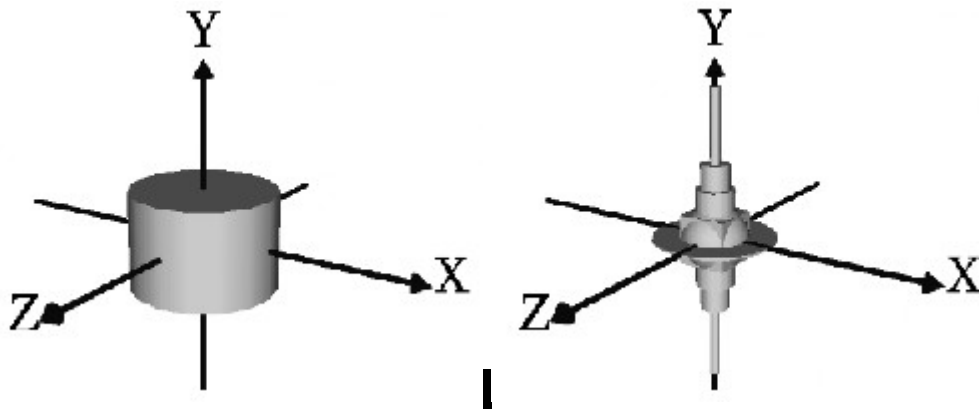
Kết quả



Hình 2.1 Ví dụ về xây dựng khung lưới và bản đồ

2.3.3 Các phép biến đổi trong VRML

Một file VRML xây dựng các thành phần của thế giới ảo. Mặc định mọi hình ảnh được xây dựng trong VRML đều được đặt ở tọa độ gốc. Biến đổi là tạo ra một hệ thống hệ trục tọa độ mới mà hệ trục này có vị trí tương đối so với hệ tọa độ mặc định. Có các phép biến đổi như: di chuyển, co giãn, xoay.



Hình 2.2 Trục XYZ với 1 hình cơ bản và 1 hình phức tạp

Nhóm Node **Transform** quản lý các phép biến đổi trong VRML như : dịch chuyển, co dãn, xoay. Node transform gồm có các node con: **translation**, **rotation**, **scale**, **children**.

❖ Phép di chuyển (Translation)

Ví dụ :

```
Transform {
  translation 2.0 0.0 0.0
  children [ . . . ]
}
```

Tham số :

➤ translation X Y Z

❖ Phép xoay (Rotation)

Ví dụ :

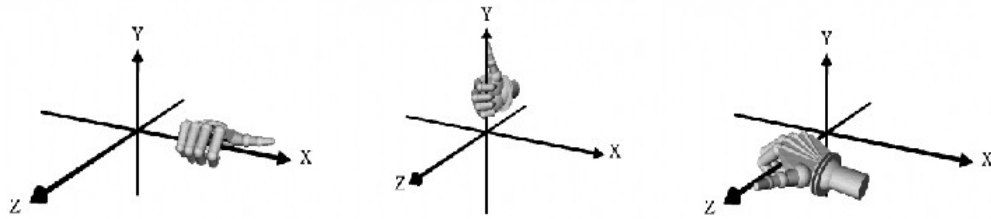
```
Transform {
  rotation 0.0 0.0 1.0 0.52
  children [ . . . ]
}
```

Tham số :

➤ rotation X Y Z Góc

➤ Góc = radians = degrees / 180.0 * 3.141

Lưu ý: để dễ dàng trong việc xoay đối tượng theo đúng chiều ta có quy tắc bàn tay phải.



Hình 2.3 Quay theo trục X, Y, Z

Nắm bàn tay phải lại, choãi ngón cái ra theo như hình vẽ. Ngón tay cái chỉ phương của trục quay thì chiều từ cổ tay đến các ngón tay chỉ chiều dương khi quay đối tượng.

❖ **Scale : co dãn đối tượng**

Ví dụ :

```
Transform {
  scale 0.5 0.5 0.5
  children [ . . . ]
}
```

Tham số :

➤ scale X Y Z

❖ **Children :** Các đối tượng được vẽ trong này sẽ chịu ảnh hưởng của các phép biến đổi trong node Transform.

Ví dụ :

```
Transform {
  translation -2.0 -1.0 0.0
  children [
    Shape {
      appearance Appearance {
        material Material { }
      }
      geometry Cylinder {
        radius 0.3
        height 6.0
      }
    }
  ]
}
```

```

    top FALSE
  }
}
|
}
Shape {
    Geometry Box { size 1.0 1.0 1.0}
}

```

Như ví dụ trên thì hình trụ sẽ dịch chuyển so với tọa độ gốc theo trục X là -2.0 và theo trục Y là -1.0, còn hình hộp thì nằm tại tọa độ gốc

2.3.4 Màu sắc trong VRML

Các đối tượng khi vẽ sẽ có mặc định là màu trắng và bạn có thể kiểm soát các thứ như : Shading color, Glow color, Transparency, Shininess, Ambient intensity. Màu sắc trong VRML được thể hiện qua 3 thông số là R G B(red-green-blue) với giá trị nằm trong khoảng từ 0.0 đến 1.0.

Node **material** nằm trong node **appearance** quản lý việc này. Node **material** có các node con là :

- ❖ **diffuseColor** : màu sắc khuyết tán
- ❖ **emissiveColor** : màu sắc của ánh sáng phát ra từ đối tượng
- ❖ **transparency** : độ trong suốt (0 : tối mờ, 1 : nét)
- ❖ **ambientIntensity** : xác định hàm lượng ánh sáng mà đối tượng phản chiếu
- ❖ **specularColor** : cho biết màu sắc của các pixel trên bề mặt của đối tượng
- ❖ **shininess** : cường độ sáng

Ví dụ :

```

Shape {
  appearance Appearance {
    material Material {
      diffuseColor 0.8 0.8 0.8
      emissiveColor 0.0 0.0 0.0
    }
  }
}

```

```

transparency 0.0
specularColor 0.71 0.70 0.56
shininess 0.16
ambientIntensity 0.4
}
}
geometry. . .
}

```

2.3.5 Nhóm node

Ta có thể nhóm các đối tượng đơn lẻ thành những đối tượng phức tạp. Việc đặt một nhóm này vào nhóm kia tạo thành một lược đồ cấu trúc các node. Một node trong nhóm có thể có bất kì các node con nào đặt bên trong trường children.

Có các node nhóm sau được hỗ trợ trong VRML: **Anchor**, **Billboard**, **Group**, **Switch**, **Transform**, **Inline**

- ❖ **Anchor**: cho phép xác định một tập hợp các đối tượng và cách liên kết đến một url, chẳng hạn như là một siêu liên kết đến thế giới VRML khác, đến một trang HTML hoặc đến một dữ liệu nào đó mà toàn bộ trình duyệt có thể đọc được. Khi kích vào một trong các đối tượng bên trong node Anchor thì toàn bộ trình duyệt sẽ đưa đến địa chỉ url. Khi đưa chuột qua đối tượng nào đó chứa trong node thì có thể nhìn thấy được địa chỉ url của nó.

Ví dụ

```

Anchor{
  Children[
    Shape{
      Geometry Sphere{}}]
  url http://www.cuasoit.com
  description "Toi trang web của lớp CT 901"
}

```

Các thuộc tính

- **children**: chứa tất cả các node bên trong nhóm

- **url**: cho biết địa chỉ url được tìm kiếm. Có thể có nhiều vị trí đặt trong url, khi đó bộ trình duyệt sẽ tìm kiếm dữ liệu trong các vị trí này theo thứ tự ưu tiên giảm dần.
 - **parameter**: cung cấp thông tin thêm vào cho bộ trình duyệt. Cho ví dụ có thể chỉ rõ là cửa sổ cần xem mà url hiển thị vào đó.
 - **description**: đưa ra một xâu, xâu này sẽ hiển thị khi người dùng di chuột qua đối tượng chứa trong node Anchor.
 - **bboxCenter**: xác định trung tâm của một hình hộp chữ nhật mà nó bao quanh tất cả các node trong nhóm. Giá trị của trường này là một điểm trong không gian.
 - **bboxSize**: xác định kích thước của hình hộp chữ nhật bao quanh tất cả các node trong nhóm. Mặc định trường này có giá trị -1 -1 -1 nghĩa là không có hình hộp nào. Tất cả các giá trị thành phần của trường này là lớn hơn hoặc bằng 0. Nếu tất cả các node con không nằm trong hình hộp thì kết quả không xác định. Hai trường bboxCenter và bboxSize là tùy chọn.
- ❖ **Group**: tạo ra một tập hợp các vật thể như là một thực thể đơn. Node này có các trường: **children**, **bboxCenter**, **bboxSize**. Các trường này tương tự như của node Anchor. Tất cả các node con trong **Group** đều hiển thị.

Ví dụ:

```
Group{
  Children[]
  bboxCenter 0 0 0
  bboxSize -1 -1 -1
}
```

- ❖ **Switch**: tạo ra một nhóm chuyển đổi. Chỉ có một node con trong nhóm được hiển thị. Và node này là do người dùng lựa chọn. Node con tuyệt đối được đánh số từ 0, không lựa chọn đánh là -1.

Ví dụ:

Các thuộc tính:

- whichChoice
 - choice
- ❖ **Transform**: xác định hệ trục tọa độ mới cho các đối tượng trong cùng một nhóm. Node này đã được trình bày ở trên.

- ❖ **Billboard:** node này tạo ra một nhóm với hệ tọa độ đặc biệt. Tất cả các node trong nhóm đều được hiển thị. Hệ tọa độ này luôn được quay về đối diện với người dùng.

Vi dụ:

```
Billboard{
  axisOfRotation 0.0 1.0 0.0
  children [...]
}
```

Các thuộc tính:

- axisOfRotationbboxCenter
- bboxSize

- ❖ **Inline:** node này sẽ được trình bày ở phần sau

2.3.6 Một số phương pháp vẽ trong VRML

Trong VRML chúng ta có thể vẽ trực tiếp đối tượng. Tuy nhiên trong nhiều trường hợp, đối tượng được sử dụng nhiều lần thì việc vẽ trực tiếp sẽ rất mất thời gian, công sức và tăng dung lượng file lên đáng kể.

Có 2 cách thông dụng nhất để giải quyết việc này đó là:

- ❖ **Inline:** hàm Inline cho phép ta gọi trực tiếp một đối tượng bên ngoài file vào file hiện tại.

Cú pháp:

```
Inline {url "Tên đường dẫn"}
```

Vi dụ:

```
Inline {url "robo.wrl"}
```

- ❖ **DEF (DEFine):** cho phép ta định nghĩa một đối tượng hay một kiểu thuộc tính

Vi dụ

```
Shape {
  appearance Appearance {
    material DEF RedColor Material {
      diffuseColor 1.0 0.0 0.0
```

```

    }
  }
  geometry. . .
}
Shape {
  appearance Appearance {
    material USE RedColor
  }
  geometry. . .
}

```

Như ví dụ trên ta định nghĩa đối tượng RedColor kiểu Material. Sau đó muốn sử dụng lại ta gọi hàm bằng từ khóa USE.

2.3.7 Texture Mapping

Ta có thể mô hình mỗi chi tiết kết cấu nhỏ bé của một đối tượng. Thay vì phải mất thời gian để vẽ ta có thể lấy một bức ảnh dán lên bề mặt kết cấu như là dán decal. Kỹ thuật này được gọi là ánh xạ kết cấu.

Trong VRML có thể sử dụng một trong các định dạng ảnh sau đây:

- ❖ GIF: 8bit, có hỗ trợ trong suốt, không là sự lựa chọn tốt cho texture mapping.
- ❖ JPEG: từ 8-16 bit, không hỗ trợ trong suốt, là sự lựa chọn tốt cho texture mapping.
- ❖ PNG: từ 8-16 bit, hỗ trợ trong suốt trên từng điểm ảnh, là sự lựa chọn tốt nhất cho texture mapping.

2.3.8 Script

Thẻ script đưa ra khả năng linh hoạt, mềm dẻo để mở rộng gần như vô hạn các yêu cầu hành động của chúng ta cũng như hành động mong đợi phản hồi từ thế giới ảo của chúng ta. Thẻ script cho phép chúng ta xác định cách tương tác với thế giới ảo bằng việc sử dụng các ngôn ngữ lập trình như Java, Javascript hoặc VRML script mới đây được đưa ra bởi Silicon Graphics, Inc.

Trong một thẻ Script có thể có các trường hay các thuộc tính, nhưng không giống với các thẻ khác, trong thẻ script chúng ta có thể xác định các sự kiện mà thẻ này có thể nhận và gửi sự kiện. Một thẻ script sẽ xác định một hành động nào đó mỗi lúc nó nhận một sự kiện, trong mỗi hành động thẻ script có thể tạo ra nhiều sự kiện. Hơn nữa thẻ script có thể được dùng để xây dựng các thủ tục khởi tạo (initiazation procedures) và dừng, tắt (shutdown procedures).

Một thẻ script có thể nhận bất kỳ các sự kiện, các giá trị mà các sự kiện này có được là các biến thuộc loại eventIn. Tên của các biến này là tên của các sự kiện mà thẻ script nhận sự kiện đó. Các biến này là chỉ có thể đọc nghĩa là chúng ta không thể thiết lập giá trị trực tiếp cho bất kỳ biến nào thuộc loại này. Một thẻ script cũng tạo ra bất kỳ các sự kiện nào, các giá trị mà các sự kiện này có được là các biến thuộc loại eventOut.

Cú pháp

```
Script
{
url[]
directOutput FALSE
mustEvaluate FALSE
eventIn Datatype EventName
eventOut Datatype EventName
field Datatype FieldName InitialValue
}
```

- url cho biết chương trình script được xử lý. Các ngôn ngữ gần đây hỗ trợ cho việc xây dựng các chương trình script này là: JAVA, JavaScript, VRMLScript.
- Có hai cách mà có thể thay đổi thế giới ảo bằng việc sử dụng script: thứ nhất là sử dụng một biến eventOut và đặt sự kiện đến thẻ khác, thứ hai là truy cập trực tiếp các trường của thẻ khác, chú ý là cả hai cách này đều phải thực hiện trong ther script.
- Điều khó khăn cho bộ trình duyệt là vừa phải vẽ thế giới ảo vừa phải xử lý các sự kiện ở cùng một thời gian.

CHƯƠNG 3. Ứng dụng VRML trong việc xây dựng xóm trọ Hoàng Trường

3.1 Bài toán

Bây giờ trên các thành thị lớn, nơi tập trung các trường đại học cao đẳng mọc lên rất nhiều, nhu cầu thuê trọ của sinh viên là rất cao nên dựa vào lợi thế đó các hộ dân gần các trường đã xây dựng lên những xóm trọ hết sức khang trang và rộng rãi, đáp ứng đúng nhu cầu của sinh viên. Chính vì vậy chúng em quyết định chọn đề tài sử dụng công nghệ thực tại ảo để mô phỏng 1 trong rất nhiều xóm trọ để sinh viên có thể tưởng tượng ra được cảnh xóm trọ như thế nào

Xóm trọ được thể hiện trên nền một trang web nên mọi người ở nhiều nơi có thể tham gia trực tiếp vào xóm trọ ảo mà không cần phải cài đặt gì nhiều.

3.2 Yêu cầu đặt ra và hướng giải quyết

3.2.1 Xây dựng xóm trọ

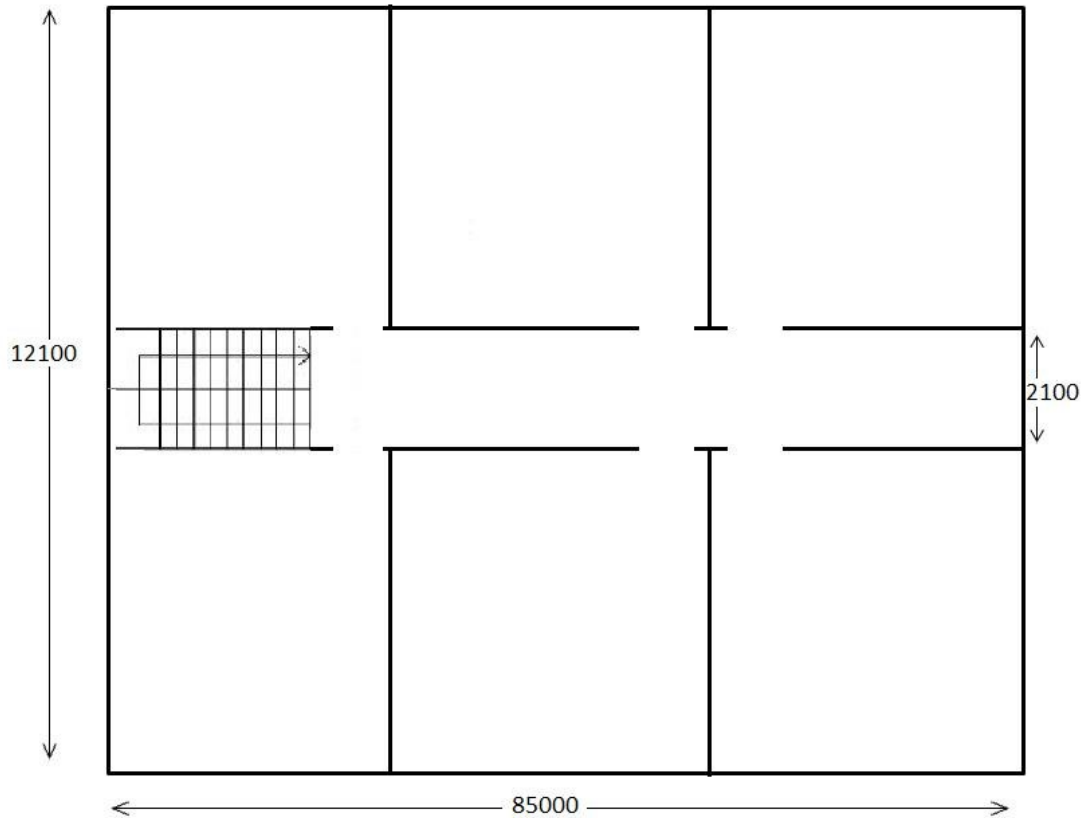
❖ Vấn đề đặt ra

Xóm trọ cần xây dựng được chia làm nhiều khu vực, gồm có:

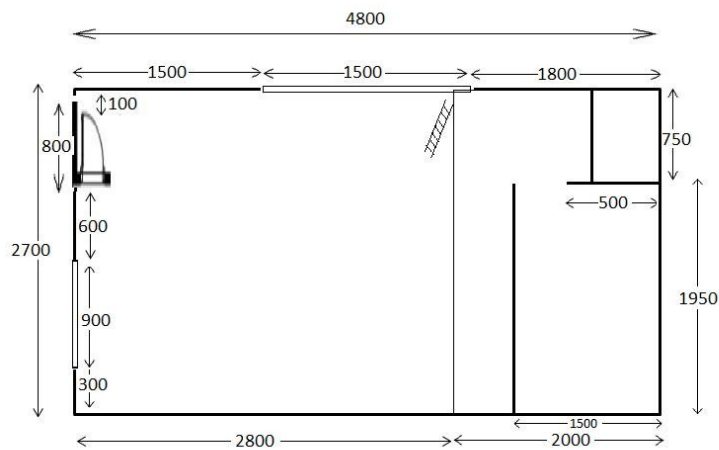
- Các phòng trọ
- Khu để xe

❖ Hướng giải quyết

Ngôn ngữ VRML có hỗ trợ việc vẽ các hình cơ bản như: hộp, cầu, nón...Vì thế nhờ vào việc sử dụng các hình cơ bản này ta có thể dễ dàng xây dựng được bộ khung của xóm trọ.



Hình 3.1 Sơ đồ xóm trọ



Hình 3.2 Sơ đồ chi tiết phòng trọ

3.2.2 Trang trí xóm trọ

❖ Vấn đề đặt ra

Xóm trọ không chỉ là một bộ khung đơn giản mà cần phải được trang trí một cách tỉ mỉ. Có như vậy người dùng mới có được cảm giác giống như một xóm

Trọ thật.

Hướng giải quyết

Ta có thể làm được điều này với texture mapping. Gán vật liệu lên các vật thể tương ứng như sàn, tường hay trần ta sẽ thu được các kết quả như ý muốn.

3.2.3 Xây dựng các đối tượng bên trong xóm trọ

❖ Vấn đề đặt ra

Trong xóm trọ phải có người, các vật dụng sinh hoạt,... Và các tác phẩm này sẽ được bài trí một cách đẹp mắt giúp người dùng dễ quan sát cũng như có cảm giác giống như một xóm trọ thật

❖ Hướng giải quyết

VRML chỉ hỗ trợ vẽ những dạng hình học cơ bản còn các dạng phức tạp thì rất khó có thể thực hiện được. Vì thế cần dùng một phần mềm bên ngoài để giải quyết việc này. Và phần mềm em chọn ở đây là 3DS Max. Các đối tượng sẽ được vẽ một cách chi tiết bằng 3DS Max và chuyển sang ngôn ngữ VRML (chương trình 3DS Max có hỗ trợ export sang đuôi .wrl). Lúc này ta chỉ cần đặt đoạn code này vào vị trí mong muốn là đối tượng đó sẽ xuất hiện trong xóm trọ.

Hình 3.3 Export sang file VRML

3.2.4 Di chuyển bên trong xóm trọ

❖ Vấn đề đặt ra

Sẽ là thiếu sót nếu như ta không thể di chuyển được trong một thế giới ảo. Xóm trọ ảo cũng vậy, người dùng có thể đi lại bên trong xóm trọ một cách linh hoạt, thoải mái để có thể quan sát được hoạt động bên trong xóm trọ. Người dùng có thể quay trái, quay phải, tiến, lùi, thậm chí là tăng tốc và giảm tốc độ.

❖ Hướng giải quyết

Có sẵn một cơ chế điều khiển được tích hợp sẵn trong ngôn ngữ VRML với chuột và các phím mũi tên. Ta chỉ cần kết hợp nó với góc quay của camera thì ta sẽ có cảm giác là đang đi lại vậy. Thực ra ở đây xóm trọ và các vật thể bên trong nó được xem như là một đối tượng và khi ta điều khiển thì chỉ có xóm trọ là di chuyển. Nhưng nhờ kết hợp

góc quay nên ta có cảm giác khi tiến lại gần thì vật to ra, và khi ra xa thì vật nhỏ lại giống như là đang di chuyển vậy.

❖ Mã nguồn

```
Viewpoint
{
  #góc nhìn
  orientation 0.024791 0.99938 -0.024992 1.57938
  #tọa độ để đặt góc nhìn
  position 4.79336 1.25 -4.7879
}
```

3.2.5 Chọn đối tượng

❖ Vấn đề đặt ra

Khi người sử dụng đi lại bên trong xóm trọ, họ có thể nhìn thấy các tác phẩm nghệ thuật ở đây như tượng, tranh, phù điêu,...Để có thể quan sát một cách kĩ càng và tỉ mỉ người sử dụng có thể kích chuột vào vật mà mình chọn. Sau đó ta sẽ tới một không gian khác mà ở đây người sử dụng có thể phóng to, thu nhỏ, xoay một cách linh hoạt để xem xét vật mà mình đã chọn một cách tỉ mỉ. Ngoài ra còn có thông tin về tác giả và tác phẩm được cung cấp cho người dùng ngay trên tác phẩm.

❖ Hướng giải quyết

Ta sẽ sử dụng một node của ngôn ngữ VRML có tác dụng tạo ra một liên kết tới file HTML hoặc VRML khác. Lúc này khi di chuyển vào vật đã được sử dụng node này, con trỏ chuột sẽ thay đổi và biến thành hình bàn tay. Khi ta kích vào sẽ tới trang HTML hoặc file VRML mà ta muốn.

❖ Mã nguồn minh họa

```
Anchor {
  # Địa chỉ mà ta muốn đến
  url "cuasoit.com"
  # Dòng thông tin sẽ hiện lên khi ta di chuột và đối tượng
  description "Twisty Stairs"
  children [ . . . ]
```

}

3.2.6 Ánh sáng

❖ Vấn đề đặt ra

Để làm cho mọi thứ trở nên thật giống như ở ngoài đời thì ta cần phải có ánh sáng. Ánh sáng sẽ làm các đối tượng 3D trở nên sống động hơn, giúp người dùng hứng thú hơn khi đi tham quan xóm trọ.

❖ Hướng giải quyết

Trong VRML có nhiều loại ánh sáng như: ánh sáng điểm, ánh sáng định hướng. Ta sẽ sử dụng 2 loại này để làm thành các nguồn sáng từ các đèn bên trong xóm trọ.

❖ Mã nguồn

```
DEF DL1 DirectionalLight
{
    direction 0.8 0.5 1.2
}
```

3.2.7 Đóng mở cửa

❖ Vấn đề đặt ra

Xóm trọ gồm có 20 phòng chính. Vì thế sẽ có các cửa để ngăn cách các phòng này với nhau. Người dùng muốn đi vào phòng nào thì phải mở cửa phòng đấy.

❖ Hướng giải quyết

Ta phải dùng một đoạn Script nhúng vào trong chương để điều khiển việc đóng mở cửa. Việc này sẽ diễn ra trong thời gian thực.

3.2.8 Tối ưu khung hình

❖ Vấn đề đặt ra

Với việc sử dụng 3DS Max để vẽ các đối tượng bên trong xóm trọ sẽ làm cho các file này trở nên rất nặng. Vì thế khi có quá nhiều đối tượng được vẽ sẽ làm cho khung hình giảm xuống chậm trọng, gây giật, chậm hình ảnh và sẽ rất khó khăn để người dùng quan sát.

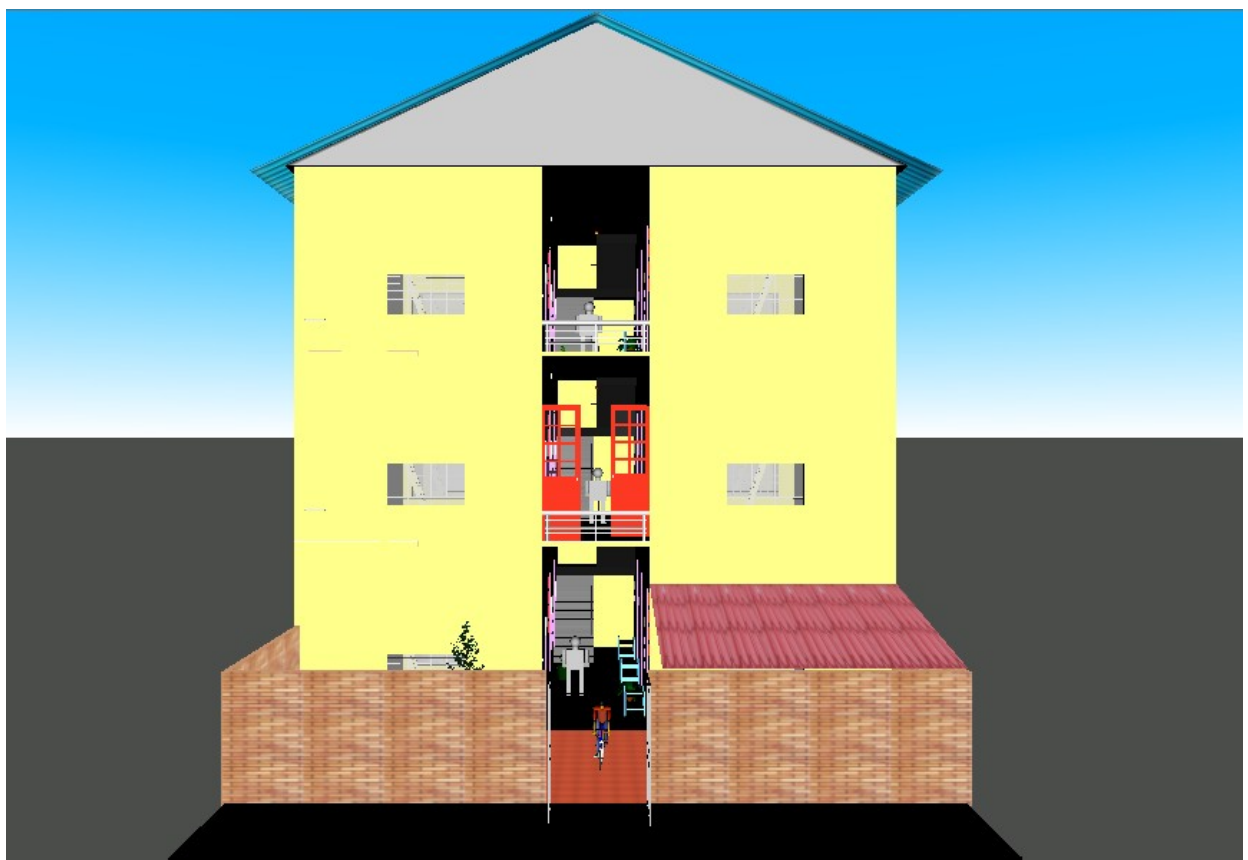
❖ Hướng giải quyết

Mỗi khu vực, phòng khác nhau sẽ được chia thành nhiều file thay vì một file như trước. Khi ta ở phòng này mà muốn đến phòng khác thì phải click chuột vào thứ gì đó (giả sử là một cánh cửa), khi đó máy tính sẽ gọi đến file vẽ phòng kia và phòng này sẽ được xóa đi. Việc này sẽ giúp loại bỏ đi các đối tượng ở những phòng khác nhưng vẫn được máy tính vẽ ra.

3.3 Kết quả đạt được và hướng phát triển

3.3.1 Kết quả

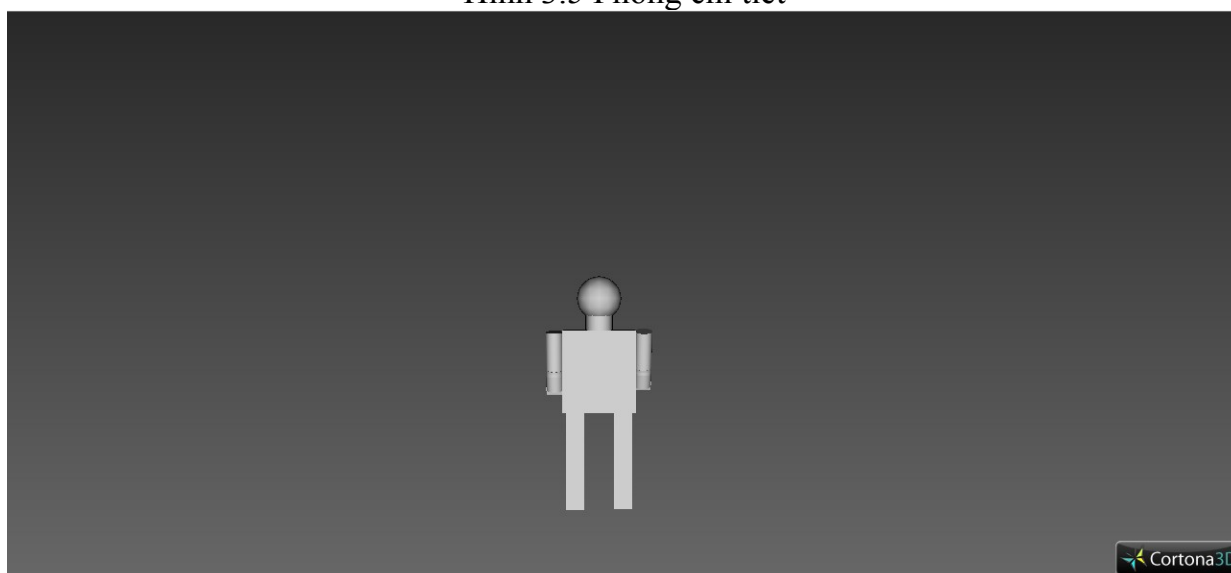
Từ việc áp dụng ngôn ngữ VRML ta thu được các kết quả sau



Hình 3.4 Toàn cảnh xóm trọ



Hình 3.5 Phòng chi tiết



Hình 3.6 Đối tượng tượng

PHẦN KẾT LUẬN

Ngày nay với sự phát triển của khoa học kỹ thuật đã mang lại những hiệu quả to lớn đặc biệt là tin học. Ở trong bất kì lĩnh vực nào của cuộc sống tin học cũng có mặt và hiệu quả của nó là một điều mọi người mong đợi. Đặc biệt trong lĩnh vực thực tế ảo là lĩnh vực còn khá mới mẻ nhưng có tầm quan trọng rất lớn.

Cũng giống như các lĩnh vực khác của công nghệ thông tin, thực tế ảo có ngôn ngữ riêng của mình đó chính là VRML. Đồ án này nhằm mục đích tìm hiểu rõ hơn về ngôn ngữ này. Qua tìm hiểu, nghiên cứu cũng như tham khảo đề tài của khóa trên và một số mã nguồn mở trên mạng nên đề tài của em đã đạt được một số mục đích:

- Tìm hiểu tổng quan về thực tại ảo
- Tìm hiểu về ngôn ngữ thực tại ảo VRML
- Xây dựng được chương trình ứng dụng

Có thể nói rằng thực tại ảo đóng một vai trò hết sức quan trọng bởi những vấn đề khó khăn mà nếu không có thực tại ảo thì rất khó để giải quyết. Nhưng khi ứng dụng thực tại ảo vào thì vấn đề đó sẽ trở nên đơn giản hơn với chi phí và thời gian ít hơn.

Tuy nhiên vì thời gian làm bài của chúng em có hạn nên đề tài còn rất nhiều thiếu sót. Mong các thầy và các bạn góp ý.

Chúng em xin chân thành cảm ơn!

TÀI LIỆU THAM KHẢO

- [1]. Lê Tấn Hùng, Huỳnh Quyết Thắng, Kỹ thuật đồ hoạ, Nhà xuất bản Khoa học và Kỹ thuật – 2000, 2002, 2004
- [2]. <http://tecfa.unige.ch/guides/vrml/pointers.html>
- [3]. <http://www.lighthouse3d.com/vrml/tutorials.shtml>